CHAPTER 9

Expressions

So far, we've been fairly cavalier in our use of expressions in Ruby. After all, `a = b + c` is pretty standard stuff. That said, Ruby expressions are different than what you might see in JavaScript or Java, and there's a lot of power and flexibility there.

One of the first differences with Ruby is that anything that can reasonably return a value does: just about everything is an expression. What does this mean in practice?

Well, for one thing, we have the ability to chain statements together:

```
a = b = c = 0
[ 3, 1, 7, 0 ].sort.reverse # => [7, 3, 1, 0]
```

Also, code structures that are statements in languages like JavaScript or Java are expressions in Ruby. For example, the `if` and `case` statements both return the value of the last expression executed:

```
song_type = if song.mp3_type == MP3::Jazz
              if song.written < Date.new(1935, 1, 1)
                Song::TradJazz
              else
                Song::Jazz
              end
            else
              Song::Other
            end

rating = case votes_cast
         when 0...10 then Rating::SkipThisOne
         when 10...50 then Rating::CouldDoBetter
         else
           Rating::Rave
         end
```

We'll talk more about `if` and `case` later on page 153.

Operator Expressions

Ruby has the basic set of operators (+, -, *, /, and so on) as well as a few surprises.